

Урок 23. Корутины. Практика. Scope, LiveData.

16 октября 2020

В этом уроке рассматриваем билдер `liveData` и операторы для получения `scope`: `viewModelScope`, `lifecycleScope`, `MainScope`.

Scope

В уроке 7 мы обсуждали, что все корутины начинаются со `scope`. Он позволяет запускать корутины и отменять их, когда это необходимо. Практические уроки также начнем с этой темы.

Рассмотрим, какие в Android есть варианты создания или получения `scope`. А также обратим внимание на то, кто отвечает за отмену `scope` и, тем самым, за жизненный цикл корутин. А после этого разберем как работать с View из корутин.

viewModelScope

`viewModelScope` - это самый популярный способ получения `scope`, который сейчас (в 2020) можно встретить в статьях и мануалах. Т.к. гугл топит за MVVM и предлагает для этого использовать `ViewModel`, то он встроил туда интеграцию с корутинами.

Как мы знаем, жизненный цикл `ViewModel` (модели) заканчивается, когда будет закрыт экран, к которому она была привязана. Корутины, которые мы запускаем в модели, должны иметь тот же жизненный цикл, чтобы они не продолжали свою работу после закрытия экрана.

Чтобы достичь этого, нам надо привязать жизненный цикл `scope` к жизненному циклу модели. Т.е. создавать `scope` при создании модели и отменять его (`cancel`) по окончании жизни модели (в методе `onCleared`). И чтобы нам самим не приходилось делать это в каждой нашей модели, существует готовое решение - `viewModelScope`.

Необходимо добавить dependency:

```
1 | androidx.lifecycle:lifecycle-viewmodel-ktx
```

В любом месте `ViewModel` мы можем использовать `viewModelScope`, чтобы запустить новую корутину:

```
1 | viewModelScope.launch {  
2 |     // ...  
3 | }
```

Что представляет собой этот `scope` изнутри?

Заглянув в исходники, увидим следующий код:

```
1 | CloseableCoroutineScope(SupervisorJob() + Dispatchers.Main.immediate))
```

`CloseableCoroutineScope` - это просто обертка над `CoroutineScope`. Она обеспечивает отмену скопа, когда модель завершает свою работу.

В качестве `Job` используется `SupervisorJob`. О нем мы говорили в Уроке 13. В отличие от обычного `Job()`, он не будет отменять все свои корутины, если одна из них завершилась с ошибкой. Т.е. корутины независимы друг от друга.

В качестве диспетчера используется `Dispatchers.Main.immediate`. Сначала давайте разберемся, что за `immediate` такой. А потом выясним, почему `Main`.

immediate

Диспетчер `Main.immediate` немного отличается от `Main` диспетчера. Это можно понять на простом примере.

Есть такой код:

```
1 | log("before")  
2 |  
3 | scope.launch {  
4 |     log("launch")  
5 | }  
6 |  
7 | log("after")
```

Он вызывается в `Main` потоке. И в зависимости от используемого в `scope` диспетчера, лог будет различным.

Если в `scope` используется диспетчер `Main`, то мы в логах получим такой результат:

```
before  
after  
launch
```

Это нормальное поведение корутины, к которому мы привыкли. Билдер `launch` создает корутину и отдает ее диспетчеру. Диспетчер кладет корутину в очередь заданий, откуда она будет взята в работу одним из потоков этого диспетчера, т.е. потоком `Main`, когда он освободится. В данный момент поток `Main` занят выполнением кода нашего примера.

В итоге код корутины не выполняется сразу же в том месте, где билдер создает корутину. Это происходит чуть позже. Именно поэтому в логах идет сначала `after`, а потом `launch`.

А если мы используем диспетчер Main.immediate, то лог будет следующий:

```
before
launch
after
```

Билдер launch также создает корутину и отдает ее диспетчеру immediate. Но диспетчер сначала проверяет, в каком потоке все это (работа билдера) происходит. Он видит, что билдер работает в потоке Main. Но именно этот поток используется Main диспетчером для выполнения кода корутин. Диспетчер понимает, что нет смысла из Main потока помещать корутину в очередь, чтобы потом тот же Main поток взял ее оттуда в работу. Можно выполнить код здесь и сейчас, т.к. мы уже в Main потоке.

Это поведение немного похоже на поведение диспетчера Unconfined, который мы обсуждали в Уроке 11. Рекомендую прочесть этот урок, чтобы лучше понимать работу диспетчеров.

Почему Main

Теперь давайте разберемся, почему в viewModelScope используется диспетчер Main, а не IO или Default. Ведь это корутины и они, вроде как, используются для выполнения фоновых задач.

Без использования корутин весь наш код в ViewModel обычно выполняется в Main потоке. Это удобно и потокобезопасно. Не нужна никакая синхронизация. Если есть потребность выполнить код в другом потоке (сеть, БД), то мы обычно выносим это в репозитории, usecase и т.п. А результат нам приходит в Main потоке (спасибо Rx оператору observeOn).

viewModelScope с Main диспетчером предлагает нам не менять наши привычки и продолжать писать код в корутинах, которые будут выполнены в Main потоке. Зачем тогда вообще нужны корутины? Чтобы была возможность вместо колбэков или конструкций RxJava использовать suspend функции. Они могут быть запущены только в корутинах. Именно suspend функции и будут тем, кто пойдет выполнять свою работу в фоновый поток, не блокируя при этом Main поток, а лишь приостанавливая код корутины. А когда результат будет готов, выполнение корутины в Main потоке возобновится.

Пример

Рассмотрим простой пример создания корутины с помощью viewModelScope.

```
1  class MainViewModel : ViewModel() {
2
3      init {
4          log("launch")
5          viewModelScope.launch {
6              while (true) {
7                  delay(1000)
8                  log("work")
9              }
10         }
11     }
12 }
13 }
```

Корутина создается при создании модели, и каждую секунду пишет в лог слово work. Она будет работать, пока ее (или ее scope) не отменят. На всякий случай уточню, что корутину можно создавать в любом методе в любое время, а не только в init при создании модели.

delay - это suspend функция, которая прервет работу корутины, как только мы эту корутину отменим. Поэтому нам тут можно самим не выполнять проверку isActive.

Давайте смотреть как поведет себя эта корутина при закрытии экрана. В Activity я добавил логирование методов onCreate и onDestroy.

Запускаем приложение и через несколько секунд закрываем его:

```
Activity onCreate
launch
work
work
work
work
Activity onDestroy, isFinishing = true
```

Корутина работает (work), только пока живет Activity. Когда Activity закрывается, оно тянет за собой ViewModel. Модель в свою очередь отменяет viewModelScope. А это уже приводит к отмене корутины, запущенной из этого scope.

Это очень удобно. Нам практически ничего не надо делать. Главное, если у вас долгорботающая корутина, то убедитесь, что вы в ней отслеживаете статус отмены (isActive или вызов cancellable suspend функции).

Проверим, что случится с этой же корутиной при повороте экрана:

```
Activity onCreate
launch
work
work
work
Activity onDestroy, isFinishing = false
Activity onCreate
work
work
work
work
...
```

Activity создается, корутина запускается (launch) и начинает работу (work). Далее при повороте экрана Activity пересоздается, а корутина продолжает работать без пересоздания, потому что модель в момент поворота продолжает жить, и viewModelScope не отменяется.

Также корутина продолжит работать если свернуть приложение.

lifecycleScope

Если добавить к проекту dependency:

```
1 | androidx.lifecycle:lifecycle-runtime-ktx
```

то станет доступно расширение:

```
1 | LifecycleOwner.lifecycleScope
```

Оно позволяет для объекта LifecycleOwner получить scope с тем же жизненным циклом. Т.е. когда LifecycleOwner перейдет в статус Destroyed, то отменится и scope. Таким LifecycleOwner объектом может быть Activity, фрагмент или View-составляющая фрагмента (viewLifecycleOwner).

Внутри этого scope тот же набор компонентов, что и в viewModelScope:

```
1 | SupervisorJob() + Dispatchers.Main.immediate
```

Т.е. это независимые друг от друга корутины и Main поток.

Простой пример использования:

```
1 | class MainActivity : AppCompatActivity() {
2 |
3 |     override fun onCreate(savedInstanceState: Bundle?) {
4 |         super.onCreate(savedInstanceState)
5 |         setContentView(R.layout.activity_main)
6 |
7 |         lifecycleScope.launch {
8 |             while (true) {
9 |                 delay(1000)
10 |                log("work")
11 |            }
12 |        }
13 |
14 |    }
15 |
16 | }
```

Таким образом можно в любом месте Activity (или фрагмента) вызвать корутину, если это необходимо. При закрытии экрана scope будет отменен. При повороте он также будет отменен, но будет создан новый в новом Activity, и новая корутина будет запущена.

lifecycleScope может быть использован в MVP для вызова корутин в презентере (если конечно вы умышленно не держите презентер живым при поворотах экрана)

MainScope

Еще одна возможность создания scope - это MainScope.

Пример:

```
1 | val scope = MainScope()
```

Под капотом похожий набор компонентов:

```
1 | ContextScope(SupervisorJob() + Dispatchers.Main)
```

Но его главное отличие от двух предыдущих - он не привязан ни к чему.

viewModelScope был привязан к модели. lifecycleScope - к LifecycleOwner. И нам не надо было думать о том, когда отменять эти scope. Они автоматически отменялись объектом, к которому они привязаны.

А MainScope - сам по себе, и может быть создан и использован где угодно. Но нам надо самим следить за тем, когда его отменять. Т.е. если у нас есть какой-то свой объект с жизненным циклом и ему надо запускать корутины, то мы можем при его создании создавать ему MainScope, а при его уничтожении - отменять этот scope.

Свой Scope

Если вам не подходят все вышеописанные scope, то всегда можно создать свой с помощью CoroutineScope. И в нем самому указать Job нужного типа, диспетчер, Handler и прочее.

```
1 | val scope = CoroutineScope(Job() + Dispatchers.Main + handler)
```

Мы это уже делали в прошлых уроках.

Также этот вариант подходит, когда нам в нашем объекте надо создать несколько scope с одним жизненным циклом, но разным диспетчером или типом Job, например.

Главное - это привязка к какому-либо Lifecycle, чтобы корутины не оставались без присмотра. Хотя я видел примеры создания глобального scope, который был один на все приложение и использовался для различных вещей не привязанных к экрану, типа уведомлений.

Работа с View

Давайте снова вернемся к теме Main потока. Я думаю, многие еще воспринимают корутины - только как фоновые потоки. И поэтому возникает вопрос, как работать с View, если мы находимся в корутине? Ведь вся работа с View должна быть в Main потоке.

Вот именно для этого и используется Main диспетчер в рассмотренных нами ранее scope. Весь код корутины выполняется в Main потоке. Кроме suspend функций. Они свою работу вынесут в фоновый поток, и как только все сделают, вернутся в Main. При этом никакой блокировки Main потока не происходит. Об этом я рассказывал в самых первых уроках этого курса.

Поэтому мы свободно можем работать с View в этих корутинах. И для нас ничего особо не меняется в этом плане. Презентер вызывает методы его View. А модель вызывает setValue (а не postValue) для своих LiveData.

Небольшой пример работы с LiveData в корутине в модели:

```
1 class MainViewModel: ViewModel() {
2
3     private val _liveData = MutableLiveData<String>()
4     val liveData = _liveData as LiveData<String>
5
6     init {
7         viewModelScope.launch {
8             while (true) {
9                 delay(1000)
10                _liveData.setValue("work")
11            }
12        }
13    }
14 }
15 }
```

Каждую секунду в LiveData передается значение work.

В следующих уроках мы рассмотрим более сложные примеры с Room, Retrofit, репозиториями и useCase. А пока нам нужно закрепить понимание того, что корутина выполняется в Main потоке и это нормально.

liveData

Есть еще один механизм, который я хотел бы рассмотреть в этом уроке. Это комбинация корутины и LiveData.

Необходимо подключить dependency:

```
1 | androidx.lifecycle:lifecycle-livedata-ktx
```

Пример использования:

```
1 val liveData = LiveData<String> {
2     while (true) {
3         delay(1000)
4         emit("work")
5     }
6 }
```

Билдер LiveData создает корутину и возвращает LiveData. Внутри корутины мы используем метод emit, чтобы постить данные в получившуюся LiveData.

Давайте разберем ключевые особенности этой пары.

Время жизни создаваемой корутины зависит от наличия подписчика создаваемой LiveData. Т.е. под капотом создается scope, который завязан на состояние активности LiveData. Корутина стартует не в момент создания, а когда кто-либо подписывается на LiveData.

Если подписчик решает отписаться когда корутина еще работает, то корутине дается таймаут 5 секунд на завершение своей работы. Если она не успеет закончить, то будет отменена. И когда кто-то снова подпишется на LiveData, корутина будет перезапущена.

Если корутина нормально (без отмены по таймауту) заканчивает свою работу, то она уже не будет перезапущена при следующей подписке на LiveData.

Т.е. можно сказать, что эта корутина - одноразовая. Успешно она выполнится только один раз. И у нас нет возможности ее перезапустить вручную чтобы, например, получить свежие данные. Надо будет создавать новую пару LiveData + корутина.

Под капотом scope все тот же набор SupervisorJob и Dispatchers.Main.immediate. Но в билдер можно передать и свой контекст. Например, чтобы сменить диспетчера.

Кратко рассмотрим, как LiveData корутина (созданная в ViewModel) реагирует на основные lifecycle события Activity, если работа этой корутины еще не завершена.

Закрытие экрана. При этом событии подписчик отпишется от LiveData. Корутина еще 5 секунд работает, а потом отменяется.

Поворот экрана. Старый подписчик отписывается от LiveData, но тут же подписывается новый в создаваемом Activity. Состояние без подписчика явно будет длиться меньше чем 5 секунд, поэтому корутина не будет отменена, а продолжит свою работу с новым подписчиком LiveData.

Сворачивание приложение больше, чем на 5 секунд. Подписчик отписывается от LiveData. Через 5 секунд корутина отменяется. Когда приложение снова будет открыто, возобновится и подписка. И т.к. корутина была отменена по таймауту, то она будет перезапущена. При этом у нас есть возможность внутри корутины узнать, какое последнее значение было отправлено (emit) перед таймаутом. Для этого используем `latestValue`.